

Bài 3: Quản lý mô hình học máy (ML Model Management)

- Yêu cầu: Class cha MLModel với các thuộc tính như tên mô hình, siêu tham số,

trạng thái huấn luyện. Class con LinearRegressionModel, RandomForestModel triển khai phương thức trừu tượng train() và predict().

- Sinh viên phải sử dụng thuộc tính riêng tư cho trọng số mô hình, thuộc tính

tính toán để đánh giá accuracy trên tập validation

```
In [1]: import abc
from typing import Any, Dict

import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
In [2]: class DatasetLoader:
        """
        Chuẩn bị dữ liệu train/val/test
        Dùng iris có sẵn trong sklearn
        """
        def __init__(self):
            self.X_train = None
            self.X_val = None
            self.X_test = None
            self.y_train = None
            self.y_val = None
            self.y_test = None

        def load_iris(self, test_size: float = 0.2, val_size: float = 0.25, random_state: int = None):
            iris = datasets.load_iris()
            X = iris.data
            y = iris.target

            X_temp, self.X_test, y_temp, self.y_test = train_test_split(
                X, y, test_size = test_size, random_state = random_state, stratify = y
            )

            self.X_train, self.X_val, self.y_train, self.y_val = train_test_split(
                X_temp, y_temp, test_size = val_size, random_state = random_state, stratify = y
            )

            return self
```

```
In [6]: from sklearn.metrics import accuracy_score

class MLModel(abc.ABC):
    """
```

```

Class cha (trường tượng) cho các mô hình sau
- __weights: thuộc tính private cho trọng số mô hình
- __X_val, __y_val: private data để tính accuracy cho validation
"""

def __init__(self, name: str, hyperparams: Dict[str, Any] | None = None):
    self.name = name
    self.hyperparams: Dict[str, Any] = hyperparams or {}
    self.trained: bool = False

    self.__weights: Any = None
    self.__X_val: np.ndarray | None = None
    self.__y_val: np.ndarray | None = None

@abc.abstractmethod
def train(
    self,
    X_train: np.ndarray,
    y_train: np.ndarray,
    X_val: np.ndarray | None = None,
    y_val: np.ndarray | None = None,
) -> None: pass

@abc.abstractmethod
def predict(self, X: np.ndarray) -> np.ndarray:
    raise NotImplementedError

@property
def validation_accuracy(self) -> float:
    """ Tính accuracy trên validation mỗi lần được truy cập """

    if self.__X_val is None or self.__y_val is None:
        raise ValueError("Validation rỗng")

    y_pred = self.predict(self.__X_val)

    # nếu là LinearRegression, làm tròn về nhãn gần nhất
    if not np.issubdtype(y_pred.dtype, np.integer):
        y_pred_labels = np rint(y_pred).astype(int)

    else:
        y_pred_labels = y_pred
    return float(accuracy_score(self.__y_val, y_pred_labels))

@property
def weights(self) -> Any:
    return self.__weights

def _set_weights(self, w: Any) -> None:
    self.__weights = w

def _set_validation_data(self, X_val: np.ndarray, y_val: np.ndarray) -> None:
    self.__X_val = X_val
    self.__y_val = y_val

```

```

In [7]: class LinearRegressionModel(MLModel):
def __init__(self, hyperparams: Dict[str, Any] | None = None):
    super().__init__(name="LinearRegressionModel", hyperparams=hyperparams)
    self.__model: LinearRegression | None = None

```

```

def train(
    self,
    X_train: np.ndarray,
    y_train: np.ndarray,
    X_val: np.ndarray | None = None,
    y_val: np.ndarray | None = None,
) -> None:
    self.__model = LinearRegression(**self.hyperparams)
    self.__model.fit(X_train, y_train)
    self.trained = True

    self._set_weights(getattr(self.__model, "coef_", None))
    if X_val is not None and y_val is not None:
        self._set_validation_data(X_val, y_val)

def predict(self, X: np.ndarray) -> np.ndarray:
    if not self.trained or self.__model is None:
        raise ValueError("LinearRegressionModel chưa được huấn luyện")
    return self.__model.predict(X)

class RandomForestModel(MLModel):
    def __init__(self, hyperparams: Dict[str, Any] | None = None):
        super().__init__(name = "RandomForestModel", hyperparams=hyperparams)
        self.__model: RandomForestClassifier | None = None

    def train(
        self,
        X_train: np.ndarray,
        y_train: np.ndarray,
        X_val: np.ndarray | None = None,
        y_val: np.ndarray | None = None,
    ) -> None:
        self.__model = RandomForestClassifier(**self.hyperparams)
        self.__model.fit(X_train, y_train)
        self.trained = True

        self._set_weights(getattr(self.__model, "feature_importances_", None))
        if X_val is not None and y_val is not None:
            self._set_validation_data(X_val, y_val)

    def predict(self, X: np.ndarray) -> np.ndarray:
        if not self.trained or self.__model is None:
            raise ValueError("RandomForestModel chưa được huấn luyện")
        return self.__model.predict(X)

```

```

In [ ]: def main():
    loader = DatasetLoader().load_iris()
    X_train, y_train = loader.X_train, loader.y_train
    X_val, y_val = loader.X_val, loader.y_val
    X_test, y_test = loader.X_test, loader.y_test

    print("Kích thước tập train:", X_train.shape, y_train.shape)
    print("Kích thước tập val: ", X_val.shape, y_val.shape)
    print("Kích thước tập test: ", X_test.shape, y_test.shape)
    print("-" * 50)

    lr_model = LinearRegressionModel()
    lr_model.train(X_train, y_train, X_val, y_val)

```

```

print(f"[{lr_model.name}]")
print("  Validation accuracy:", lr_model.validation_accuracy)

y_test_pred_lr = lr_model.predict(X_test)
y_test_pred_lr_labels = np rint(y_test_pred_lr).astype(int)
test_acc_lr = accuracy_score(y_test, y_test_pred_lr_labels)
print("  Test accuracy      :", test_acc_lr)
print("  Weights (coef_):    ", lr_model.weights)
print("-" * 50)

rf_model = RandomForestModel(hyperparams={"n_estimators": 200, "random_state"
rf_model.train(X_train, y_train, X_val, y_val)

print(f"[{rf_model.name}]")
print("  Validation accuracy:", rf_model.validation_accuracy)

y_test_pred_rf = rf_model.predict(X_test)
test_acc_rf = accuracy_score(y_test, y_test_pred_rf)
print("  Test accuracy      :", test_acc_rf)
print("  Feature importances:", rf_model.weights)

if __name__ == "__main__":
    main()

```

Kích thước tập train: (90, 4) (90,)

Kích thước tập val: (30, 4) (30,)

Kích thước tập test: (30, 4) (30,)

[LinearRegressionModel]

Validation accuracy: 0.9

Test accuracy : 0.9666666666666667

Weights (coef_): [-0.15930435 0.00745384 0.19067979 0.766461]

[RandomForestModel]

Validation accuracy: 0.9333333333333333

Test accuracy : 0.9333333333333333

Feature importances: [0.11895255 0.01207888 0.41320645 0.45576212]